frontend.md

Project: AI Business Automation Platform
Component: Frontend Server
Status: Final Requirements, Design, and Implementation Plan
Last Updated: 2026-07-07
Primary Goal: Build a professional, secure, responsive SaaS frontend that allows users to manage AI agents, workflows, tools, approvals, knowledge, memory, evaluations, audits, and organization settings.

---

1. Purpose

The frontend server exists to deliver the user-facing web application for the platform.

It provides the interface where users can:

- Sign in and manage sessions.
- View the main dashboard.
- Create, inspect, and manage AI agents.
- Configure tools and integrations.
- Build and run workflows.
- Review workflow approvals.
- Manage knowledge sources and indexed documents.
- Inspect memory records.
- View evaluations and quality metrics.
- Monitor process runs.
- Review audit logs.
- Manage organization settings, users, billing, security, and API keys.

The frontend server is responsible for:

- Presentation.
- Interaction.
- Routing.
- Layout.
- UI state.
- Frontend validation.
- User experience.
- Client-side real-time updates.
- Design-system implementation.

The frontend server must **not** contain core backend business logic.

---

2. Product Vision

The frontend should feel like a serious enterprise SaaS application for operating AI-powered business automation.

It should not look like a generic AI demo.

The interface should communicate:

- Trust.
- Reliability.
- Operational clarity.
- Security.
- Professionalism.
- Speed.
- Control.
- Observability.

Users should always understand:

- What agents exist.
- What workflows exist.
- What is running.
- What failed.
- What needs approval.
- What knowledge sources are connected.
- What data was used.
- What actions were taken.
- Who did what.
- What requires attention.

---

3. Core Design Principle

The frontend must be designed through a deliberate design process.

Trae must use **OpenDesign MCP** before implementing major page layouts.

The workflow should be:

```
Trae
  -> OpenDesign MCP
    -> Search design references
    -> Select a suitable design system
    -> Extract tokens and layout guidance
    -> Produce a layout plan
    -> Implement React/TypeScript/Tailwind frontend
```

The frontend must not be designed only from generic AI memory.

---

4. Scope

4.1 In Scope

The frontend server must include:

- Public entry page or redirect behavior.
- Authentication pages.
- Authenticated app shell.
- Dashboard page.
- Agents section.
- Tools section.
- Workflows section.
- Workflow run detail section.
- Approvals section.
- Knowledge section.
- Memory section.
- Evaluations section.
- Processes section.
- Audit logs section.
- Organization settings.
- User settings.
- API key management UI.
- Security settings UI.
- Billing placeholder or billing page if backend supports it.
- Responsive layouts.
- Loading states.
- Empty states.
- Error states.
- Permission-aware navigation.
- Typed API integration.
- Real-time workflow run UI.
- OpenDesign-based design documentation.

4.2 Out of Scope

The frontend must not implement:

- Workflow execution logic.
- Agent execution logic.
- Tool execution logic.
- Permission enforcement as the only source of truth.
- Database writes directly.
- Background jobs.
- Embedding/indexing logic.
- Secret storage.
- Provider API key handling in browser code.
- Audit log creation.
- Billing calculations.
- Authentication verification as the final authority.

The backend remains the source of truth for:

- Authentication.
- Authorization.
- Permissions.
- Workflow execution.
- Agent execution.
- Tool execution.
- Knowledge indexing.
- Memory storage.
- Evaluation execution.
- Governance rules.
- Audit logging.
- Organization security.

---

5. Recommended Technology Stack

5.1 Framework

Use:

```
Next.js
React
TypeScript
Tailwind CSS
```

Recommended architecture:

```
Browser
  -> Next.js Frontend Server
    -> Backend API
      -> FastAPI backend services
```

5.2 Package Manager

Recommended:

```
pnpm
```

Acceptable alternatives:

```
npm
yarn
```

5.3 Styling

Use:

```
Tailwind CSS
Design tokens
Reusable UI components
```

The frontend should not use random hardcoded colors.

Colors, typography, spacing, border radius, shadows, and motion rules should come from the design system generated or selected through OpenDesign.

5.4 Forms

Recommended:

```
React Hook Form
Zod
```

Form rules:

- Validate client-side for usability.
- Backend remains the final validator.
- Display server validation errors clearly.
- Disable submit during pending requests.
- Prevent duplicate submissions.

5.5 API Client

Use a typed API client generated from the backend OpenAPI schema.

Recommended options:

```
openapi-typescript
orval
openapi-fetch
custom generated TypeScript client
```

The frontend must not guess backend request and response shapes manually.

5.6 Real-Time Updates

Use:

```
SSE for workflow run updates
WebSocket only if bidirectional live interaction is required
```

SSE is preferred for:

- Workflow run timeline updates.
- Logs streaming.
- Approval waiting states.
- Step status updates.
- Process monitoring.

5.7 Testing

Use:

```
Playwright for end-to-end tests
Vitest or Jest for unit tests
React Testing Library for component tests
axe or equivalent accessibility checks
```

5.8 Deployment

The frontend should be deployable without Docker.

Recommended deployment style:

```
pnpm install
pnpm build
pnpm start
```

Production process managers may include:

```
systemd
PM2
managed Node hosting
Vercel-like platform
```

---

6. Runtime Responsibilities

6.1 Frontend Server Responsibilities

The frontend server handles:

- Rendering pages.
- Routing users.
- Protecting frontend routes.
- Calling backend APIs.
- Managing UI session state.
- Displaying backend data.
- Handling frontend errors.
- Displaying real-time updates.
- Presenting role-aware navigation.
- Applying design tokens.
- Serving static assets.
- Providing accessible UI behavior.

6.2 Backend Responsibilities

The backend handles:

- User authentication validation.
- Authorization.
- Roles and permissions.
- Agent persistence.
- Workflow persistence.
- Workflow execution.
- Tool execution.
- Approval state transitions.
- Audit log writes.
- Knowledge ingestion.
- Memory writes.
- Evaluation jobs.
- Organization governance.
- Secrets and credentials.

6.3 Boundary Rule

The frontend may request actions.

The backend decides whether the action is allowed.

Example:

```
Frontend:
  User clicks "Run Workflow"

Backend:
  Verifies permission
  Creates run
  Starts execution
  Emits status events
  Writes audit log
```

The frontend must not assume that a hidden button is sufficient security.

---

7. Application Architecture

7.1 High-Level Architecture

```
User Browser
  |
  | HTTPS
  v
Next.js Frontend Server
  |
  | HTTPS / internal network
  v
Backend API
  |
  +-- Auth service
  +-- Agents service
  +-- Tools service
  +-- Workflows service
  +-- Runs service
  +-- Approvals service
  +-- Knowledge service
  +-- Memory service
  +-- Evaluation service
  +-- Audit service
  +-- Organization service
```

7.2 Frontend Rendering Strategy

Use a hybrid rendering approach:

- Server-render app shell where useful.
- Client-render highly interactive widgets.
- Use client components for:
  - Command palette.
  - Realtime run timeline.
  - Modals.
  - Drawers.
  - Data tables with filters.
  - Workflow builder.
  - Logs viewer.
- Use server components for:
  - Static layouts.
  - Initial data fetch where cookies can be passed securely.
  - Metadata.

- Basic page composition.

7.3 Route Protection

Protected routes:

```
/app/*
```

Rules:

- If user is not authenticated, redirect to `/login`.
- If user lacks organization access, redirect to organization selection or access-denied page.
- If user lacks permission for a route, display `403 Access Denied`.
- Route protection is UX-level only.
- Backend authorization remains final.

---

8. OpenDesign MCP Requirement

8.1 Mandatory Requirement

Before Trae creates or significantly modifies any major frontend page layout, Trae must call the MCP server named:

```
opendesign
```

Major page layouts include:

- Dashboard.
- Agents list.
- Agent detail.
- Workflow list.
- Workflow builder.
- Workflow detail.
- Workflow run detail.
- Approvals queue.
- Knowledge dashboard.
- Evaluation dashboard.
- Audit logs.
- Settings.
- Billing.
- Security pages.

8.2 Required OpenDesign MCP Calls

For every major page, Trae must run the following MCP process:

```
1. get_director_protocol
2. search_designs
3. get_design_system
4. fetch_design_spec_markdown
5. Summarize layout plan
6. Implement page
```

8.3 Required Design Output From Trae

Before writing code, Trae must produce:

```
- Selected design reference
- Why the reference fits the page
- Extracted design tokens
- Typography approach
- Spacing approach
- Layout grid
- Component hierarchy
- Responsive behavior
- Accessibility concerns
- Implementation plan
```

8.4 Fallback Rule

If OpenDesign MCP is unavailable:

1. Trae must stop.
2. Trae must report that OpenDesign MCP is unavailable.
3. Trae must not continue with a generic layout unless the project owner explicitly approves a fallback.
4. If fallback is approved, Trae must document the reason in:

```
frontend/docs/design/open-design-reference.md
```

---

9. OpenDesign MCP Setup for Trae

9.1 MCP File Location

Store the MCP server file here:

```
frontend/.mcp/opendesign-mcp.mjs
```

or at project root:

```
.mcp/opendesign-mcp.mjs
```

Recommended project-root setup:

```
mkdir -p .mcp
```

Download the official OpenDesign MCP server file from the official OpenDesign source and save it as:

```
.mcp/opendesign-mcp.mjs
```

9.2 Trae MCP Configuration

Create:

```
.trae/mcp.json
```

Use an absolute path.

Example for macOS or Linux:

```
{
  "mcpServers": {
    "opendesign": {
      "command": "node",
      "args": ["/ABSOLUTE/PATH/TO/YOUR/PROJECT/.mcp/opendesign-mcp.mjs"]
    }
  }
}
```

Example for Windows:

```
{
  "mcpServers": {
    "opendesign": {
      "command": "node",
      "args": ["C:\\ABSOLUTE\\PATH\\TO\\YOUR\\PROJECT\\.mcp\\opendesign-mcp.mjs"]
    }
  }
}
```

After creating or editing this file:

```
Restart Trae.
```

9.3 MCP Security Rules

The OpenDesign MCP server should be treated as a privileged development tool.

Rules:

- Use the official source only.
- Pin or record the downloaded file version if possible.
- Do not give MCP servers unnecessary secrets.
- Do not expose production API keys to MCP tools.
- Review any MCP-generated layout or code before committing.
- Do not allow MCP tools to mutate unrelated project files without approval.
- Document selected design references.
- Do not copy proprietary assets directly.
- Use references for layout inspiration, not asset theft.

---

10. Trae Project Rules

Create:

```
.trae/project_rules.md
```

Use the following content:

```
# Frontend Project Rules

## Mandatory OpenDesign MCP Usage

Before implementing or redesigning any major frontend page layout, call the MCP
server named `opendesign`.

Required MCP workflow:
1. Call `get_director_protocol`.
2. Call `search_designs`.
3. Select a suitable SaaS, dashboard, developer-tool, workflow, or enterprise
reference.
4. Call `get_design_system`.
5. Call `fetch_design_spec_markdown`.
6. Extract design tokens, spacing, typography, layout structure, motion guidance,
and component hierarchy.
7. Produce a short layout plan before writing code.
8. Implement the page using React, TypeScript, and Tailwind CSS.

Do not create generic dashboard UI from memory when OpenDesign MCP is available.
```

```
## Frontend Architecture Rules

- Use TypeScript.
- Use Next.js App Router.
- Use Tailwind CSS.
- Use reusable components.
- Use semantic HTML.
- Use accessible components.
- Use typed backend API clients.
- Do not manually guess backend API shapes.
- Do not place backend business logic in frontend components.
- Do not expose secrets in browser code.
- Do not store sensitive auth tokens in localStorage.
- Prefer HttpOnly secure cookies for sessions.
- Add loading, empty, and error states for every major page.
- All pages must be responsive.

## Design Rules

- Use OpenDesign-derived tokens.
- Do not hardcode random colors.
- Keep visual hierarchy clear.
- Prioritize enterprise SaaS clarity.
- Use status colors consistently.
- Make destructive actions obvious and confirmed.
- Preserve keyboard navigation.
- Preserve focus states.
- Ensure color contrast.
- Avoid generic AI-themed gradients unless justified by the selected design
system.

## Implementation Rules

- Keep files small and focused.
- Extract reusable UI primitives.
- Extract page-specific feature components.
- Add tests for critical flows.
- Run type checking before completion.
- Run linting before completion.
- Document assumptions.
- If backend endpoints are missing, create clear TODOs and API contract notes
instead of inventing fake APIs.
```

---

11. Recommended Trae Prompts

11.1 General Dashboard Prompt

```
Use the OpenDesign MCP server before writing code.
```

```
I am building the frontend for an AI business automation platform.

Design the main authenticated dashboard layout.

Requirements:
- Professional SaaS dashboard
- Left sidebar navigation
- Top command/search area
- Agent status cards
- Workflow run activity
- Approvals queue
- Knowledge base health
- Evaluation score summary
- Audit/security indicators
- Responsive desktop/tablet/mobile behavior
- Clean enterprise look
- Not generic AI dashboard style

First:
1. Call OpenDesign get_director_protocol.
2. Search OpenDesign for a suitable SaaS, AI, developer-tool, or workflow-
dashboard reference.
3. Retrieve the design system and design spec.
4. Summarize the layout plan.
5. Then implement the page using React, TypeScript, and Tailwind CSS.
```

11.2 Workflow Detail Prompt

```
Use OpenDesign MCP to design the /app/workflows/[workflowId] page.

The page should include:
- Workflow title and metadata
- Workflow status
- Visual workflow steps
- Run button
- Latest run status
- Approval checkpoints
- Logs panel
- Error panel
- Version history
- Settings drawer

Call OpenDesign first, select a strong reference, summarize the layout plan, then
implement the page.
```

11.3 Workflow Run Detail Prompt

```
Use OpenDesign MCP to design the /app/workflow-runs/[runId] page.
```

```
The page should include:
- Run status summary
- Timeline of steps
- Live logs
- Tool calls
- Agent messages
- Approval waiting state
- Error details
- Retry action
- Cancel action
- Final output panel
- Audit metadata

Use a professional operations-console style. Call OpenDesign first.
```

11.4 Knowledge Page Prompt

```
Use OpenDesign MCP to design the /app/knowledge page.

The page should include:
- Knowledge health summary
- Connected sources
- Recently indexed documents
- Failed ingestion jobs
- Search box
- Source type filters
- Index freshness indicators
- Add source action
- Document detail drawer

Use OpenDesign before implementation.
```

---

12. Information Architecture

12.1 Main Routes

```
/
  Public landing page or redirect

/login
/register
/forgot-password
/reset-password
/accept-invite

/app
  Main dashboard
```

```
/app/agents
/app/agents/new
/app/agents/[agentId]

/app/tools
/app/tools/[toolId]

/app/workflows
/app/workflows/new
/app/workflows/[workflowId]

/app/workflow-runs/[runId]

/app/approvals
/app/approvals/[approvalId]

/app/knowledge
/app/knowledge/sources
/app/knowledge/sources/[sourceId]
/app/knowledge/documents
/app/knowledge/documents/[documentId]
/app/knowledge/search

/app/memory
/app/memory/[memoryId]

/app/evaluations
/app/evaluations/[evaluationId]
/app/evaluations/runs/[evaluationRunId]

/app/processes
/app/processes/[processId]

/app/audit-logs

/app/settings
/app/settings/organization
/app/settings/users
/app/settings/roles
/app/settings/billing
/app/settings/api-keys
/app/settings/security
/app/settings/integrations
/app/settings/profile
```

12.2 Navigation Groups

Sidebar navigation should be grouped:

```
Main
  Dashboard
```

```
  Agents
  Workflows
  Approvals

Data
  Knowledge
  Memory

Quality
  Evaluations
  Processes

Security
  Audit Logs

Admin
  Settings
```

12.3 Global Header

The authenticated app header should include:

- Breadcrumbs.
- Global search or command button.
- Environment indicator if not production.
- Organization switcher.
- Notifications.
- User menu.

12.4 Command Palette

Shortcut:

```
Cmd/Ctrl + K
```

Actions:

- Create agent.
- Create workflow.
- Search knowledge.
- Open recent workflow run.
- Review approvals.
- Invite user.
- Open API keys.
- Open audit logs.
- Open security settings.
- Start evaluation.
- Add knowledge source.

13. User Roles and Permissions

The frontend should support role-aware UI.

Suggested roles:

```
Owner
Admin
Developer
Operator
Reviewer
Viewer
Billing Manager
Security Auditor
```

13.1 Permission Examples

```
agents:read
agents:create
agents:update
agents:delete

tools:read
tools:create
tools:update
tools:delete

workflows:read
workflows:create
workflows:update
workflows:delete
workflows:run

runs:read
runs:cancel
runs:retry

approvals:read
approvals:approve
approvals:reject

knowledge:read
knowledge:create
knowledge:update
knowledge:delete
knowledge:index

memory:read
memory:delete
```

```
evaluations:read
evaluations:create
evaluations:run

audit_logs:read

settings:read
settings:update

users:read
users:invite
users:update
users:remove

billing:read
billing:update

api_keys:read
api_keys:create
api_keys:revoke

security:read
security:update
```

13.2 Permission Display Rules

The frontend should:

- Hide actions users cannot perform.
- Show disabled states when useful.
- Show clear explanations for disabled actions.
- Display access-denied pages for forbidden routes.
- Never rely on hidden UI as security.

---

14. Core Layout Design

14.1 App Shell

The authenticated app must use a persistent shell.

Required elements:

```
- Sidebar
- Header
- Main content region
- Breadcrumb
- Command palette trigger
- Organization switcher
- User menu
```

```
- Notification center
- Responsive mobile navigation
```

14.2 Desktop Layout

Desktop should use:

```
Left sidebar + top header + content region
```

Suggested behavior:

- Sidebar fixed or sticky.
- Main content scrolls.
- Header sticky.
- Content width adapts by page.
- Dense data pages may use full width.
- Detail pages may use two-column layouts.

14.3 Tablet Layout

Tablet should use:

- Collapsible sidebar.
- Header remains visible.
- Tables become card lists when needed.
- Important actions move into overflow menus.

14.4 Mobile Layout

Mobile should use:

- Drawer navigation.
- Bottom-safe spacing.
- Single-column content.
- Cards instead of wide tables.
- Sticky primary action where appropriate.
- Logs and timelines optimized for vertical scrolling.

---

15. Visual Design Requirements

15.1 Visual Style

The frontend should feel:

- Modern.
- Enterprise-grade.
- Calm.

- Precise.
- Trustworthy.
- Operational.
- Data-rich but not cluttered.

Avoid:

- Random gradients.
- Toy-like AI visuals.
- Oversized empty cards.
- Excessive animations.
- Inconsistent status colors.
- Unclear hierarchy.

15.2 Design Tokens

Create token files:

```
frontend/src/design/tokens.ts
frontend/src/design/theme.ts
frontend/docs/design/open-design-reference.md
```

Token categories:

```
Color
Typography
Spacing
Radii
Shadows
Borders
Surfaces
Status colors
Motion
Z-index
Breakpoints
```

15.3 Status Colors

Use consistent semantic status colors:

```
Success
Warning
Error
Info
Neutral
Running
Pending
```

```
Paused
Cancelled
Draft
Published
```

Example status usage:

```
Workflow run succeeded -> Success
Workflow run failed -> Error
Workflow waiting approval -> Warning or Pending
Agent disabled -> Neutral
Tool unavailable -> Error
Knowledge source indexing -> Running
```

15.4 Typography

Typography should support:

- Page titles.
- Section headings.
- Card titles.
- Metric labels.
- Table text.
- Code/log text.
- Metadata.
- Empty state copy.
- Error copy.

Use monospace typography for:

- IDs.
- Logs.
- API keys.
- JSON snippets.
- Tool call payloads.
- Trace IDs.

15.5 Motion

Motion should be subtle.

Use motion for:

- Drawer open/close.
- Command palette.
- Toasts.
- Skeleton loading.
- Status transitions.

- Timeline updates.

Avoid motion that distracts from operational monitoring.

---

16. Page Requirements

16.1 Public Root Page

Route:

```
/
```

Purpose:

- Show public landing page or redirect authenticated users to `/app`.

Requirements:

- If authenticated, redirect to dashboard.
- If unauthenticated, show simple product intro or redirect to `/login`.
- Must be lightweight.
- Must not expose internal app data.

16.2 Login Page

Route:

```
/login
```

Purpose:

- Allow users to sign in.

Required UI:

- Email field.
- Password field.
- Submit button.
- Forgot password link.
- Register link if registration enabled.
- Error message area.
- Loading state.
- Organization invite handling if applicable.

States:

```
Default
Submitting
Invalid credentials
Account locked
MFA required
Server unavailable
```

Security rules:

- Do not store tokens in localStorage.
- Use secure cookie session flow if backend supports it.
- Do not reveal whether an email exists unless backend policy allows it.

16.3 Register Page

Route:

```
/register
```

Purpose:

- Allow new user or organization registration if enabled.

Required UI:

- Name.
- Email.
- Password.
- Organization name.
- Terms checkbox if required.
- Submit button.
- Login link.

Rules:

- If public registration disabled, show invite-only message.
- Validate password client-side for usability.
- Backend performs final validation.

16.4 Dashboard Page

Route:

```
/app
```

Purpose:

- Provide operational overview.

Required sections:

```
- Welcome/header summary
- Agent health cards
- Workflow run activity
- Pending approvals
- Knowledge base health
- Evaluation summary
- Recent audit events
- Process status
- Quick actions
```

Recommended dashboard cards:

```
Active Agents
Workflows Running
Pending Approvals
Failed Runs
Knowledge Sources
Evaluation Pass Rate
Security Alerts
Recent Tool Errors
```

Primary actions:

- Create agent.
- Create workflow.
- Add knowledge source.
- Review approvals.
- Run evaluation.

Empty state:

- If no agents/workflows exist, show onboarding checklist.

Dashboard good idea:

```
Add an onboarding checklist:
1. Create first agent
2. Add a tool
3. Add knowledge source
4. Create workflow
5. Run workflow
6. Review results
```

16.5 Agents List Page

Route:

```
/app/agents
```

Purpose:

- Show all agents in the organization.

Required UI:

- Page title.
- Create agent button.
- Search.
- Filters.
- Agent cards or table.
- Status indicators.
- Last run information.
- Tool count.
- Knowledge access indicator.
- Owner or creator.
- Updated date.

Filters:

```
Status
Capability
Tool access
Knowledge access
Owner
Created date
```

Agent statuses:

```
Draft
Active
Disabled
Error
Archived
```

Empty state:

```
No agents yet.
Create your first agent to automate a business task.
```

16.6 Create Agent Page

Route:

```
/app/agents/new
```

Purpose:

- Create a new AI agent.

Required UI:

- Agent name.
- Description.
- Instructions.
- Model/provider selection if supported.
- Tool permissions.
- Knowledge access.
- Memory settings.
- Safety settings.
- Test prompt area.
- Save draft button.
- Create agent button.

Validation:

- Name required.
- Instructions required.
- Tool permissions must be explicit.
- Dangerous tool access should show warning.

Good idea:

```
Add a guided setup wizard:
1. Identity
2. Instructions
3. Tools
4. Knowledge
5. Safety
6. Test
```

16.7 Agent Detail Page

Route:

```
/app/agents/[agentId]
```

Purpose:

- Inspect and manage one agent.

Required sections:

```
- Agent header
- Status
- Description
- Instructions
- Tools
- Knowledge access
- Memory settings
- Recent runs
- Evaluation results
- Audit activity
- Danger zone
```

Actions:

- Edit.
- Duplicate.
- Enable/disable.
- Run test.
- View runs.
- Delete/archive.

States:

- Loading agent.
- Agent not found.
- Access denied.
- Agent archived.

16.8 Tools List Page

Route:

```
/app/tools
```

Purpose:

- Manage tools and integrations available to agents/workflows.

Required UI:

- Tool catalog.
- Connected tools.
- Tool status.
- Required credentials indicator.
- Permission scope.
- Usage count.
- Last used.
- Error status.

Tool categories:

```
Communication
CRM
Documents
Data
Internal API
Web
Files
Code
Human Approval
```

Security rule:

- Never display secret values.
- Only show masked credential metadata.

16.9 Tool Detail Page

Route:

```
/app/tools/[toolId]
```

Purpose:

- View and configure a tool.

Required sections:

- Tool overview.
- Configuration.
- Credentials status.

- Permission scopes.
- Agents using this tool.
- Workflows using this tool.
- Recent calls.
- Failure rate.
- Audit events.

Actions:

- Connect.
- Reconnect.
- Disable.
- Test tool.
- Rotate credentials.
- Remove.

16.10 Workflows List Page

Route:

```
/app/workflows
```

Purpose:

- View and manage workflows.

Required UI:

- Workflow list.
- Create workflow button.
- Search.
- Filters.
- Status.
- Trigger type.
- Last run status.
- Success rate.
- Owner.
- Updated date.

Workflow statuses:

```
Draft
Active
Paused
Error
Archived
```

Filters:

```
Status
Trigger type
Owner
Last run status
Created date
Updated date
```

Empty state:

```
No workflows yet.
Build your first workflow to automate a repeatable process.
```

16.11 Create Workflow Page

Route:

```
/app/workflows/new
```

Purpose:

- Build a new workflow.

Required UI:

- Workflow name.
- Description.
- Trigger selection.
- Step builder.
- Agent step.
- Tool step.
- Condition step.
- Approval step.
- Notification step.
- Test run.
- Save draft.
- Publish.

Good idea:

```
Support workflow templates:
- Customer support triage
- Invoice processing
```

```
- Lead enrichment
- Document review
- Compliance approval
- Daily report generation
```

16.12 Workflow Detail Page

Route:

```
/app/workflows/[workflowId]
```

Purpose:

- View, edit, run, and monitor a workflow.

Required sections:

```
- Header
- Status
- Metadata
- Visual step map
- Trigger configuration
- Step list
- Latest run summary
- Approval checkpoints
- Version history
- Settings
- Audit activity
```

Actions:

- Run now.
- Edit.
- Duplicate.
- Pause.
- Publish.
- Archive.
- View latest run.
- Create new version.

Critical UI behavior:

- Running a workflow should show immediate feedback.
- If backend creates a run, redirect or link to `/app/workflow-runs/[runId]`.

16.13 Workflow Run Detail Page

Route:

```
/app/workflow-runs/[runId]
```

Purpose:

- Show live workflow run execution.

This is one of the most important pages.

Required sections:

```
- Run header
- Status summary
- Timeline
- Current step
- Step details
- Live logs
- Tool calls
- Agent messages
- Input payload
- Output payload
- Approval state
- Error details
- Retry/cancel actions
- Audit metadata
```

Run statuses:

```
Queued
Running
Waiting for Approval
Succeeded
Failed
Cancelled
Timed Out
Partially Completed
```

Timeline item types:

```
Workflow started
Agent step started
Agent step completed
Tool call started
```

```
Tool call completed
Approval requested
Approval approved
Approval rejected
Condition evaluated
Error occurred
Retry started
Workflow completed
```

Real-time behavior:

- Connect to SSE endpoint.
- Show reconnecting state.
- Deduplicate events.
- Preserve scroll position.
- Allow user to pause auto-scroll.
- Show final state when run completes.
- Handle connection loss.

Actions:

- Cancel run.
- Retry failed step.
- Retry workflow.
- Approve pending step.
- Reject pending step.
- Download logs.
- Copy run ID.
- Copy trace ID.

Good idea:

```
Add an operations-style split view:
Left: timeline
Right: selected step details and logs
```

16.14 Approvals List Page

Route:

```
/app/approvals
```

Purpose:

- Allow humans to review pending approvals.

Required UI:

- Pending approvals queue.
- Search.
- Filters.
- Priority.
- Requesting workflow.
- Requesting agent.
- Created date.
- Due date.
- Risk level.
- Approve/reject actions.

Filters:

```
Status
Priority
Workflow
Agent
Risk level
Due date
Requester
```

Approval statuses:

```
Pending
Approved
Rejected
Expired
Cancelled
```

Empty state:

```
No approvals need review.
Everything is moving automatically.
```

16.15 Approval Detail Page

Route:

```
/app/approvals/[approvalId]
```

Purpose:

- Review one approval request.

Required sections:

- Request summary.
- Requested action.
- Reason.
- Input data.
- Proposed output.
- Risk indicators.
- Related workflow run.
- Agent/tool involved.
- Audit trail.
- Comment box.
- Approve button.
- Reject button.

Security requirement:

- High-risk approvals should show a stronger confirmation dialog.

16.16 Knowledge Page

Route:

```
/app/knowledge
```

Purpose:

- Show knowledge base overview.

Required sections:

```
- Knowledge health summary
- Connected sources
- Indexed documents
- Recent ingestion jobs
- Failed indexing jobs
- Search knowledge
- Add source action
```

Metrics:

```
Total sources
Indexed documents
Failed documents
Last indexed time
Index freshness
Storage usage
Search success rate
```

16.17 Knowledge Sources Page

Route:

```
/app/knowledge/sources
```

Purpose:

- Manage knowledge sources.

Required UI:

- Sources table.
- Source type.
- Status.
- Document count.
- Last sync.
- Error count.
- Add source button.

Source statuses:

```
Connected
Indexing
Failed
Paused
Disconnected
```

Source types:

```
File upload
Google Drive
Notion
Confluence
SharePoint
Website
Database
```

```
API
S3
Internal documents
```

16.18 Knowledge Search Page

Route:

```
/app/knowledge/search
```

Purpose:

- Search indexed knowledge.

Required UI:

- Search input.
- Filters.
- Results list.
- Source badges.
- Relevance score if backend provides it.
- Snippets.
- Document detail drawer.
- Copy citation/reference.

Good idea:

```
Add "Why this result?" metadata showing source, document, chunk, and last indexed
date.
```

16.19 Memory Page

Route:

```
/app/memory
```

Purpose:

- Inspect memory records used by agents.

Required UI:

- Memory records table.

- Search.
- Filters.
- Agent filter.
- User/session filter if applicable.
- Created date.
- Last used.
- Delete action if allowed.

Security rule:

- Sensitive memory values may need redaction.
- Backend decides visibility.

16.20 Evaluations Page

Route:

```
/app/evaluations
```

Purpose:

- View and run evaluations.

Required UI:

- Evaluation suites.
- Recent evaluation runs.
- Pass/fail summary.
- Score trend.
- Agent/workflow tested.
- Run evaluation button.

Metrics:

```
Pass rate
Average score
Regression count
Failed test cases
Last run date
```

Good idea:

```
Add "quality gate" indicators before publishing high-risk workflows.
```

16.21 Evaluation Detail Page

Route:

```
/app/evaluations/[evaluationId]
```

Purpose:

- View an evaluation suite.

Required sections:

- Test cases.
- Expected behavior.
- Latest results.
- Historical score.
- Failures.
- Run settings.
- Trigger evaluation.

16.22 Processes Page

Route:

```
/app/processes
```

Purpose:

- Monitor backend processes, jobs, and long-running tasks.

Required UI:

- Process list.
- Status.
- Type.
- Started time.
- Duration.
- Owner.
- Related workflow/run.
- Progress.
- Error message.

Process statuses:

```
Queued
Running
```

```
Succeeded
Failed
Cancelled
Retrying
```

16.23 Audit Logs Page

Route:

```
/app/audit-logs
```

Purpose:

- Show security and activity history.

Required UI:

- Audit table.
- Search.
- Filters.
- Date range.
- Actor.
- Action.
- Resource.
- IP/device metadata if available.
- Export action if allowed.

Audit event examples:

```
User signed in
Agent created
Workflow published
Workflow run started
Approval approved
API key created
Tool credentials rotated
User invited
Permission changed
Knowledge source deleted
```

Security rule:

- Audit logs must be read-only from the frontend.
- No frontend mutation should edit audit records.

16.24 Settings Page

Route:

```
/app/settings
```

Purpose:

- Main settings hub.

Sections:

```
Organization
Users
Roles
Billing
API Keys
Security
Integrations
Profile
```

16.25 Organization Settings

Route:

```
/app/settings/organization
```

Required UI:

- Organization name.
- Logo.
- Slug.
- Timezone.
- Default language.
- Data retention settings if supported.
- Save button.

16.26 User Management

Route:

```
/app/settings/users
```

Required UI:

- Users table.
- Invite user button.
- Role badges.
- Status.
- Last active.
- Remove user.
- Change role.

User statuses:

```
Active
Invited
Suspended
Removed
```

16.27 API Keys Page

Route:

```
/app/settings/api-keys
```

Required UI:

- API keys list.
- Create key.
- Key name.
- Scopes.
- Created date.
- Last used.
- Revoke action.

Security rules:

- Show full API key only once after creation.
- Never show full key again.
- Use masked display.
- Confirm revoke action.

16.28 Security Settings

Route:

```
/app/settings/security
```

Required UI:

- MFA settings.
- Session policy.
- Password policy.
- SSO settings if supported.
- Allowed domains.
- Audit export if supported.
- Security alerts.

---

17. Reusable Components

17.1 Layout Components

```
AppShell
Sidebar
SidebarItem
Header
Breadcrumbs
PageHeader
PageSection
ContentGrid
MobileNav
OrganizationSwitcher
UserMenu
NotificationCenter
CommandPalette
```

17.2 UI Components

```
Button
IconButton
Input
Textarea
Select
Checkbox
RadioGroup
Switch
Tabs
Card
Badge
StatusBadge
Tooltip
Popover
```

```
DropdownMenu
Dialog
Drawer
Toast
Skeleton
EmptyState
ErrorState
LoadingState
DataTable
Pagination
FilterBar
SearchInput
DateRangePicker
CodeBlock
LogViewer
Timeline
MetricCard
ProgressBar
Stepper
ConfirmDialog
```

17.3 Domain Components

```
AgentCard
AgentStatusBadge
AgentToolList
AgentRunList

WorkflowCard
WorkflowStatusBadge
WorkflowStepMap
WorkflowRunTimeline
WorkflowRunStatusHeader
WorkflowRunLogs
WorkflowVersionHistory

ApprovalCard
ApprovalRiskBadge
ApprovalDecisionPanel

KnowledgeSourceCard
KnowledgeHealthCard
KnowledgeSearchResult
DocumentDetailDrawer

EvaluationScoreCard
EvaluationRunTable
EvaluationFailureList

AuditLogTable
AuditEventBadge
```

```
ApiKeyTable
UserRoleBadge
PermissionMatrix
```

---

18. Frontend Folder Structure

Recommended structure:

```
frontend/
  app/
    layout.tsx
    page.tsx
    globals.css

    login/
      page.tsx

    register/
      page.tsx

    forgot-password/
      page.tsx

    reset-password/
      page.tsx

    accept-invite/
      page.tsx

    app/
      layout.tsx
      page.tsx

      agents/
        page.tsx
        new/
          page.tsx
        [agentId]/
          page.tsx

      tools/
        page.tsx
        [toolId]/
          page.tsx

      workflows/
        page.tsx
        new/
          page.tsx
```

```
    [workflowId]/
      page.tsx

  workflow-runs/
    [runId]/
      page.tsx

  approvals/
    page.tsx
    [approvalId]/
      page.tsx

  knowledge/
    page.tsx
    sources/
      page.tsx
      [sourceId]/
        page.tsx
    documents/
      page.tsx
      [documentId]/
        page.tsx
    search/
      page.tsx

  memory/
    page.tsx
    [memoryId]/
      page.tsx

  evaluations/
    page.tsx
    [evaluationId]/
      page.tsx
    runs/
      [evaluationRunId]/
        page.tsx

  processes/
    page.tsx
    [processId]/
      page.tsx

  audit-logs/
    page.tsx

  settings/
    page.tsx
    organization/
      page.tsx
    users/
      page.tsx
    roles/
      page.tsx
```

```
          billing/
            page.tsx
          api-keys/
            page.tsx
          security/
            page.tsx
          integrations/
            page.tsx
          profile/
            page.tsx

    src/
      components/
        ui/
        layout/
        dashboard/
        agents/
        tools/
        workflows/
        approvals/
        knowledge/
        memory/
        evaluations/
        processes/
        audit/
        settings/

      design/
        tokens.ts
        theme.ts
        status.ts

      hooks/
        use-command-palette.ts
        use-permissions.ts
        use-realtime-run.ts
        use-toast.ts

      lib/
        api/
          client.ts
          generated/
          errors.ts
        auth/
          session.ts
          permissions.ts
        config/
          env.ts
        realtime/
          sse.ts
        routes/
          paths.ts
        formatting/
          dates.ts
```

```
        status.ts
      errors/
        app-error.ts

    stores/
      command-palette-store.ts
      notification-store.ts

    types/
      domain.ts
      permissions.ts
      navigation.ts

  docs/
    design/
      open-design-reference.md
      layout-decisions.md
      design-token-map.md

    api/
      frontend-api-contract.md

  tests/
    e2e/
    unit/
    components/

  .trae/
    mcp.json
    project_rules.md

  .mcp/
    opendesign-mcp.mjs

  package.json
  tsconfig.json
  tailwind.config.ts
  next.config.js
  postcss.config.js
  playwright.config.ts
  README.md
  frontend.md
```

---

19. Environment Variables

19.1 Public Variables

Only variables prefixed with `NEXT_PUBLIC_` are available in browser code.

Allowed public variables:

```
NEXT_PUBLIC_APP_ENV
NEXT_PUBLIC_APP_NAME
NEXT_PUBLIC_API_BASE_URL
NEXT_PUBLIC_ENABLE_REGISTRATION
NEXT_PUBLIC_ENABLE_BILLING
NEXT_PUBLIC_ENABLE_SSO
```

19.2 Server-Only Variables

Server-only variables must not be exposed to the browser.

```
BACKEND_API_INTERNAL_URL
SESSION_COOKIE_NAME
NODE_ENV
```

19.3 Forbidden in Frontend

Never expose:

```
DATABASE_URL
REDIS_URL
JWT_SECRET
OPENAI_API_KEY
ANTHROPIC_API_KEY
STRIPE_SECRET_KEY
WEBHOOK_SECRET
PROVIDER_CLIENT_SECRET
INTERNAL_SERVICE_TOKEN
ENCRYPTION_KEY
```

---

20. API Integration

20.1 Typed Client Requirement

The frontend must use a typed API client.

Generated types should be stored in:

```
frontend/src/lib/api/generated/
```

API client wrapper:

```
frontend/src/lib/api/client.ts
```

20.2 API Error Handling

All API errors should map to consistent UI behavior.

Error types:

```
400 Validation error
401 Unauthorized
403 Forbidden
404 Not found
409 Conflict
422 Invalid input
429 Rate limited
500 Server error
503 Service unavailable
Network error
Timeout
```

UI responses:

```
401 -> redirect to login
403 -> show access denied
404 -> show not found
409 -> show conflict message
422 -> show form validation errors
429 -> show rate-limit retry message
500 -> show error state
Network -> show retry option
```

20.3 API Request Rules

All API calls must:

- Use the typed client.
- Include credentials when using cookie-based auth.
- Handle loading states.
- Handle errors.
- Avoid duplicate submissions.
- Cancel stale requests when appropriate.
- Avoid exposing secrets in logs.

---

21. Real-Time Workflow Run Updates

21.1 SSE Client

Create:

```
frontend/src/lib/realtime/sse.ts
frontend/src/hooks/use-realtime-run.ts
```

The hook should handle:

- Connection open.
- Message events.
- Error events.
- Reconnect attempts.
- Backoff.
- Final state close.
- Duplicate event IDs.
- Manual disconnect.

21.2 Run Event Shape

Expected frontend event model:

```ts
type WorkflowRunEvent = {
  id: string;
  runId: string;
  type:
    | "run.started"
    | "run.status_changed"
    | "step.started"
    | "step.completed"
    | "step.failed"
    | "tool_call.started"
    | "tool_call.completed"
    | "approval.requested"
    | "approval.approved"
    | "approval.rejected"
    | "log"
    | "error"
    | "run.completed"
    | "run.failed"
    | "run.cancelled";
  timestamp: string;
  payload: Record<string, unknown>;
};
```

21.3 Realtime UI Rules

The UI must:

- Show live status.

- Show connection status.
- Show reconnecting state.
- Not lose already received events.
- Avoid duplicate events.
- Allow manual refresh.
- Stop reconnecting when run reaches final state.

Final states:

```
Succeeded
Failed
Cancelled
Timed Out
```

---

22. Loading, Empty, and Error States

Every major page must include:

```
Loading state
Empty state
Error state
Permission denied state
Not found state where applicable
```

22.1 Loading State

Use skeletons for:

- Dashboard cards.
- Tables.
- Detail headers.
- Timeline rows.
- Logs panels.

22.2 Empty State

Empty states must include:

- Clear title.
- Short explanation.
- Primary action.
- Optional secondary action.
- Useful illustration or icon if consistent with design system.

Example:

```
No workflows yet
Create your first workflow to automate a repeatable business process.
[Create workflow]
```

22.3 Error State

Error states must include:

- Friendly explanation.
- Retry button if possible.
- Technical reference ID if available.
- Support/contact option if applicable.

---

23. Accessibility Requirements

The frontend must target WCAG AA-level accessibility.

Required:

- Semantic HTML.
- Keyboard navigation.
- Visible focus states.
- Sufficient color contrast.
- ARIA labels where needed.
- Accessible dialogs.
- Accessible dropdowns.
- Accessible command palette.
- Proper form labels.
- Error messages associated with fields.
- No color-only status communication.
- Reduced-motion support.
- Screen-reader-friendly loading states.

Specific requirements:

- Sidebar navigation must be keyboard navigable.
- Command palette must trap focus while open.
- Modals and drawers must restore focus when closed.
- Data tables must have meaningful headers.
- Status badges must include text, not only color.
- Timeline items must be readable by screen readers.

---

24. Security Requirements

24.1 Authentication

Preferred:

```
HttpOnly Secure SameSite cookies
```

Avoid:

```
localStorage token storage
sessionStorage token storage
browser-exposed long-lived secrets
```

24.2 Authorization

Frontend must:

- Display UI based on permissions.
- Still call backend for final authorization.
- Gracefully handle `403 Forbidden`.

24.3 XSS Protection

Rules:

- Do not render raw HTML from backend unless sanitized.
- Escape user-generated content.
- Use safe Markdown rendering if required.
- Do not inject untrusted scripts.
- Avoid `dangerouslySetInnerHTML`.

24.4 CSRF

If using cookie-based auth:

- Backend should provide CSRF protection.
- Frontend should send CSRF token header if required.

24.5 Secret Handling

The frontend must never display or log secrets.

Sensitive fields:

```
API keys
OAuth tokens
Provider credentials
Internal service tokens
Webhook secrets
Private documents
Memory records marked sensitive
```

24.6 Destructive Action Confirmation

Require confirmation for:

- Delete agent.
- Archive workflow.
- Delete knowledge source.
- Delete memory.
- Revoke API key.
- Remove user.
- Disable security feature.
- Cancel running workflow.

High-risk actions should require typing confirmation.

---

25. Performance Requirements

25.1 Page Performance

The frontend should optimize:

- Initial load.
- Route transitions.
- Dashboard rendering.
- Large tables.
- Logs rendering.
- Timeline rendering.
- Search responsiveness.

25.2 Techniques

Use:

- Code splitting.
- Lazy loading.
- Pagination.
- Virtualized lists for logs if needed.
- Debounced search.
- Request cancellation.
- Server-side filtering.
- Optimistic updates only when safe.
- Memoization for expensive UI calculations.

25.3 Large Data Rules

For large data:

- Do not load all audit logs at once.
- Do not load all workflow runs at once.

- Do not load all logs at once if backend supports pagination.
- Use cursor pagination where possible.
- Use server-side search and filters.

---

26. Observability

The frontend should include observability hooks.

Track:

- Page load failures.
- API failures.
- Realtime connection failures.
- Unhandled exceptions.
- Slow pages.
- Failed form submissions.
- Command palette usage.
- Workflow run page errors.

Do not track:

- Secrets.
- Full prompts if sensitive.
- Private documents.
- API keys.
- Raw memory content unless explicitly allowed and sanitized.

Recommended events:

```
frontend.page.viewed
frontend.api.error
frontend.form.submitted
frontend.workflow_run.opened
frontend.workflow_run.realtime_disconnected
frontend.approval.approved
frontend.approval.rejected
frontend.command_palette.opened
```

---

27. Testing Requirements

27.1 Unit Tests

Test:

- Utility functions.
- Permission helpers.
- API error mapping.

- Status formatting.
- Date formatting.
- Realtime event reducers.

27.2 Component Tests

Test:

- Buttons.
- Forms.
- Dialogs.
- Data tables.
- Status badges.
- Empty states.
- Error states.
- Workflow timeline.
- Approval panel.

27.3 End-to-End Tests

Critical E2E flows:

```
Login
Dashboard loads
Create agent
Create workflow
Run workflow
View workflow run updates
Approve pending approval
Search knowledge
Invite user
Create API key
Revoke API key
View audit logs
Logout
```

27.4 Accessibility Tests

Run accessibility checks on:

- Login.
- Dashboard.
- Workflow run detail.
- Approvals.
- Settings.
- Dialogs.
- Command palette.

---

28. Implementation Plan

Phase 0: Discovery and Contracts

Goals:

- Confirm backend API routes.
- Confirm auth model.
- Confirm permissions model.
- Confirm OpenAPI schema.
- Confirm SSE or WebSocket events.
- Confirm entity names and IDs.
- Confirm deployment target.

Deliverables:

```
docs/api/frontend-api-contract.md
docs/design/open-design-reference.md
Initial route map
Initial navigation map
```

Exit criteria:

- Backend OpenAPI schema available.
- Auth flow documented.
- Main entities documented.
- Realtime event shape documented.

Phase 1: Project Setup

Tasks:

```
1. Create Next.js TypeScript app.
2. Install Tailwind CSS.
3. Configure ESLint.
4. Configure Prettier.
5. Configure path aliases.
6. Configure environment loader.
7. Create base folder structure.
8. Add base app layout.
9. Add global CSS.
10. Add design token files.
11. Add README startup instructions.
```

Commands:

```
pnpm install
pnpm dev
pnpm build
pnpm lint
pnpm typecheck
```

Exit criteria:

- App starts locally.
- TypeScript works.
- Tailwind works.
- Lint works.
- Build succeeds.

Phase 2: Trae and OpenDesign Setup

Tasks:

```
1. Add .mcp/opendesign-mcp.mjs.
2. Add .trae/mcp.json.
3. Add .trae/project_rules.md.
4. Restart Trae.
5. Test MCP server availability.
6. Ask Trae to call get_director_protocol.
7. Document selected design process.
```

Deliverables:

```
.trae/mcp.json
.trae/project_rules.md
docs/design/open-design-reference.md
docs/design/design-token-map.md
```

Exit criteria:

- Trae can call OpenDesign MCP.
- Trae uses OpenDesign before major layouts.
- Design reference documented.

Phase 3: Design System Foundation

Tasks:

```
1. Extract design tokens from OpenDesign.
2. Create color tokens.
3. Create typography tokens.
4. Create spacing tokens.
5. Create status tokens.
6. Create base UI components.
7. Create layout components.
8. Create loading/empty/error components.
9. Create status badge system.
10. Create form components.
```

Base components to implement first:

```
Button
Input
Textarea
Select
Card
Badge
StatusBadge
Dialog
Drawer
Dropdown
Skeleton
EmptyState
ErrorState
DataTable
Toast
Tooltip
Tabs
```

Exit criteria:

- Basic UI system complete.
- Components use tokens.
- Components are accessible.
- Components are responsive.

Phase 4: Authentication and Session UI

Tasks:

```
1. Implement login page.
2. Implement register page if enabled.
3. Implement forgot password page.
4. Implement reset password page.
5. Implement invite acceptance page.
```

```
6. Implement session fetch.
7. Implement protected route handling.
8. Implement logout.
9. Implement user menu.
10. Implement unauthorized and forbidden states.
```

Exit criteria:

- User can log in.
- User can log out.
- Protected routes redirect correctly.
- App shell shows current user.
- Unauthorized API errors handled.

Phase 5: App Shell and Navigation

Tasks:

```
1. Build AppShell.
2. Build Sidebar.
3. Build Header.
4. Build Breadcrumbs.
5. Build OrganizationSwitcher.
6. Build UserMenu.
7. Build NotificationCenter placeholder.
8. Build CommandPalette.
9. Add responsive mobile navigation.
10. Add permission-aware nav filtering.
```

Exit criteria:

- `/app` routes share common shell.
- Sidebar works on desktop.
- Drawer nav works on mobile.
- Command palette opens with Cmd/Ctrl + K.
- Navigation respects permissions.

Phase 6: Dashboard

Tasks:

```
1. Use OpenDesign MCP for dashboard layout.
2. Build dashboard data fetch.
3. Build metric cards.
4. Build workflow activity widget.
5. Build approvals widget.
```

```
6. Build knowledge health widget.
7. Build evaluation summary widget.
8. Build audit/security widget.
9. Build onboarding checklist.
10. Add loading, empty, and error states.
```

Exit criteria:

- Dashboard gives clear operational overview.
- Empty organization sees onboarding.
- Data cards are responsive.
- Design documented.

Phase 7: Agents and Tools

Tasks:

```
1. Build agents list.
2. Build create agent flow.
3. Build agent detail page.
4. Build agent edit form.
5. Build tool list.
6. Build tool detail page.
7. Build tool connection states.
8. Add permission-aware actions.
9. Add loading/empty/error states.
10. Add tests for basic flows.
```

Exit criteria:

- User can view agents.
- User can create agent if permitted.
- User can inspect agent details.
- User can view tools.
- Secrets are never exposed.

Phase 8: Workflows

Tasks:

```
1. Use OpenDesign MCP for workflow pages.
2. Build workflows list.
3. Build create workflow page.
4. Build workflow detail page.
5. Build visual step map.
6. Build workflow settings drawer.
```

```
7. Build version history UI.
8. Build run workflow action.
9. Redirect to run detail after run starts.
10. Add loading/empty/error states.
```

Exit criteria:

- User can view workflows.
- User can create workflow if permitted.
- User can view workflow details.
- User can start workflow run if permitted.

Phase 9: Workflow Run Realtime UI

Tasks:

```
1. Build SSE client.
2. Build useRealtimeRun hook.
3. Build run status header.
4. Build timeline.
5. Build logs viewer.
6. Build tool call viewer.
7. Build agent message viewer.
8. Build approval waiting panel.
9. Build error details panel.
10. Build cancel/retry actions.
11. Add reconnect handling.
12. Add final state behavior.
```

Exit criteria:

- Run page updates live.
- Reconnect behavior works.
- Logs are readable.
- Timeline is clear.
- Pending approvals are obvious.
- Failed runs show useful error details.

Phase 10: Approvals

Tasks:

```
1. Build approvals queue.
2. Build approval filters.
3. Build approval detail page.
4. Build approve action.
```

```
5. Build reject action.
6. Build risk badges.
7. Build confirmation dialogs.
8. Link approvals to workflow runs.
9. Add audit metadata display.
10. Add loading/empty/error states.
```

Exit criteria:

- Reviewer can approve/reject.
- High-risk approvals are clearly marked.
- Approval decisions show confirmation.
- Empty state is useful.

Phase 11: Knowledge and Memory

Tasks:

```
1. Build knowledge overview.
2. Build knowledge sources list.
3. Build add source UI.
4. Build source detail page.
5. Build documents list.
6. Build document detail page.
7. Build knowledge search page.
8. Build memory list.
9. Build memory detail page.
10. Add redaction support.
```

Exit criteria:

- User can inspect knowledge health.
- User can search knowledge.
- User can view source status.
- Sensitive memory is handled safely.

Phase 12: Evaluations and Processes

Tasks:

```
1. Build evaluations list.
2. Build evaluation detail.
3. Build evaluation run detail.
4. Build score cards.
5. Build failure list.
6. Build processes list.
```

```
7. Build process detail page.
8. Add filters.
9. Add loading/empty/error states.
10. Add tests.
```

Exit criteria:

- User can view quality metrics.
- User can inspect failed evaluations.
- User can monitor processes.

Phase 13: Audit Logs and Settings

Tasks:

```
1. Build audit logs table.
2. Add audit filters.
3. Build settings hub.
4. Build organization settings.
5. Build users page.
6. Build roles page.
7. Build API keys page.
8. Build security settings.
9. Build billing page or placeholder.
10. Build integrations page.
```

Exit criteria:

- Admin can manage organization settings.
- API key UI is secure.
- Audit logs are readable and filterable.
- Settings respect permissions.

Phase 14: Quality, Security, and Release

Tasks:

```
1. Run full typecheck.
2. Run lint.
3. Run unit tests.
4. Run component tests.
5. Run E2E tests.
6. Run accessibility checks.
7. Review security.
8. Review performance.
9. Review responsive layouts.
10. Build production bundle.
```

```
11. Deploy staging.
12. Test staging.
13. Deploy production.
```

Exit criteria:

- Build passes.
- Tests pass.
- Critical flows pass.
- No browser-exposed secrets.
- Mobile layout works.
- Production deployment works.

---

29. Good Product Ideas to Include

29.1 Onboarding Checklist

For new organizations:

```
Create first agent
Connect first tool
Add knowledge source
Create first workflow
Run first workflow
Invite teammate
Review audit logs
```

29.2 Command Palette

Power-user navigation:

```
Cmd/Ctrl + K
```

Should support:

- Navigation.
- Creation actions.
- Recent items.
- Search.
- Admin shortcuts.

29.3 Recent Activity Feed

Dashboard should show:

- Recent workflow runs.

- Failed tool calls.
- New approvals.
- User invitations.
- Security changes.
- Knowledge syncs.

29.4 Risk Indicators

Show risk on:

- Approval requests.
- Tool calls.
- Workflow publishing.
- API key scopes.
- Security settings.

Risk levels:

```
Low
Medium
High
Critical
```

29.5 Templates

Provide templates for:

- Agents.
- Workflows.
- Evaluations.
- Knowledge sources.

29.6 Saved Views

For tables:

- Save filters.
- Save search.
- Save column visibility.
- Share view with team if backend supports it.

29.7 Explainability Drawer

For workflow runs:

```
Why did this happen?
What data was used?
Which agent made the decision?
```

```
Which tools were called?
Which approval rule applied?
```

29.8 Quality Gates

Before publishing workflows:

- Show latest evaluation score.
- Show failed tests.
- Warn if no evaluation exists.
- Warn if workflow uses high-risk tools.

---

30. Acceptance Criteria

The frontend server is complete when:

```
- Frontend starts without Docker.
- Next.js app builds successfully.
- Trae can connect to OpenDesign MCP.
- Trae calls OpenDesign before major page layouts.
- Design references are documented.
- App shell exists.
- Login flow exists.
- Protected routes work.
- Dashboard exists.
- Agents pages exist.
- Tools pages exist.
- Workflows pages exist.
- Workflow run detail page supports live status UI.
- Approvals pages exist.
- Knowledge pages exist.
- Memory pages exist.
- Evaluations pages exist.
- Processes pages exist.
- Audit logs page exists.
- Settings pages exist.
- Command palette exists.
- All major pages are responsive.
- All major pages have loading states.
- All major pages have empty states.
- All major pages have error states.
- API calls use typed client.
- Permissions affect UI.
- Backend remains final source of authorization.
- No backend secrets are exposed.
- API keys are masked.
- Destructive actions require confirmation.
- Accessibility checks pass for critical pages.
- E2E tests pass for critical flows.
```

31. Definition of Done for Each Page

A page is done only when:

```
- OpenDesign MCP was used if the page is a major layout.
- Layout plan was documented.
- Page is implemented with TypeScript.
- Page uses shared layout components.
- Page uses design tokens.
- Page is responsive.
- Page has loading state.
- Page has empty state.
- Page has error state.
- Page handles permission restrictions.
- Page uses typed API client.
- Forms validate input.
- Server errors display clearly.
- Important actions have confirmation.
- Accessibility basics are satisfied.
- Page passes lint and typecheck.
- Critical interactions are tested.
```

32. Final Frontend Rule

The frontend server should be a clean, professional SaaS interface for operating AI business automation.

The design workflow is mandatory:

```
Trae
  -> OpenDesign MCP
    -> Choose real design reference
    -> Extract design tokens
    -> Create layout plan
    -> Implement frontend page
```

The frontend must be:

```
Secure
Typed
Responsive
Accessible
Observable
Permission-aware
Operationally clear
Design-system driven
```

The frontend must not be:

```
Generic
Toy-like
Secret-leaking
Backend-logic-heavy
Unstructured
Hardcoded
Inaccessible
Unclear during failures
```